1 Abstract

The Codynamic Book Machine is an intent-driven, schema-backed, multi-agent authoring system for producing long-form technical writing as a coordinated workflow rather than a sequence of isolated drafts. It begins with author intent and outline structure, normalizes that intent into a canonical work object, and then distributes section-level labor across specialized agents that draft, review, revise, and assemble content into publication artifacts.

The central claim of the system is that a book can be managed as a living, machine-readable work graph in which structure, dependencies, messages, and generated artifacts evolve together. Instead of treating outline, prose, citations, diagrams, and build outputs as separate concerns, the Codynamic Book Machine keeps them aligned through shared schema, durable task state, and agent coordination. This makes the book both the product and the operating context of the system.

In practice, the machine supports a pipeline that starts with conversational intake or imported outline material, converts that input into a canonical representation, decomposes the work into section-sized tasks, and routes those tasks through agents responsible for drafting, reference handling, diagram generation, design, and assembly. The result is a structured authoring environment in which changes can be traced, dependencies can be managed explicitly, and publication artifacts can be regenerated from the same underlying work object.

This paper presents the architecture, data model, and coordination workflow of the Codynamic Book Machine, and uses the system itself as a working example of codynamic design: a writing system whose content and control structure are developed together.

2 Introduction

2.1 The Coordination Problem in Long-Form Writing

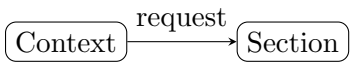
Long-form writing is not difficult because authors lack ideas; it is difficult because the work must remain coherent across many moving parts at once. An outline can change while a draft is already in progress. A section can be revised after a citation has been added elsewhere. A diagram may be requested before the surrounding explanation has stabilized. Review comments can arrive after the text has already been copied into a build. Even the compiled output can drift from the source if the repository does not clearly connect the manuscript, the references, the media assets, and the generated artifacts.

This is the coordination problem in book-scale authoring: the book is not a single file, but a system of interdependent representations. The author thinks in terms of intent, structure, and argument. The writing pipeline also has to manage section payloads, shared references, diagrams, review feedback, and build products. When those representations are maintained separately and updated at different times, inconsistencies accumulate. A chapter outline may promise one sequence of claims while the draft follows another. A citation key may be used in prose before it exists in the bibliography. A figure may be described in text but never generated. A build may succeed while still reflecting an outdated revision of the manuscript.

These failures are especially visible in collaborative or agent-assisted workflows. Different agents may work on different sections, but each agent only sees part of the whole. One agent can improve a paragraph while another changes the outline that paragraph depends on. A reference agent can register sources after a section has already been drafted. A diagram agent can produce an asset that no longer matches the latest wording. Without a shared coordination model, the system spends as much effort reconciling artifacts as it does producing them.

The problem is not merely editorial; it is structural. A book has dependencies that are semantic as well as mechanical. The meaning of one section can depend on the definitions established earlier. The validity of a claim can depend on whether a citation has been registered. The usefulness of a diagram can depend on whether the surrounding explanation still matches it. The correctness of a build can depend on whether all of these pieces were updated together. In practice, long-form writing requires continuous alignment among intent, outline, draft text, references, media, review state, and compiled output.

The Codynamic Book Machine is designed around that reality. Rather than treating the manuscript as a static document that is occasionally edited, it treats the book as a coordinated system whose parts must evolve together. The rest of this paper explains how that coordination is represented, how work is divided among agents, and how the machine keeps the outline, artifacts, and compiled outputs in sync as the book develops.



Conceptual diagram for 'The Coordination Problem in Long-Form Writing': Explain why books are difficult to manage when outlines, drafts, citations, diagrams, reviews, and builds drift apart.

Figure 1: Conceptual diagram for 'The Coordination Problem in Long-Form Writing': Explain why books are difficult to manage when outlines, drafts, citations, diagrams, reviews, and builds drift apart.

2.2 The Machine as a Codynamic System

The central claim of the Codynamic Book Machine is that a book is not produced by a single linear pass from outline to prose. It is produced by a coordinated system in which the outline, the agents, the dependency graph, the section artifacts, and the compiled outputs remain in motion together. Each part of the system both constrains and informs the others, so the project stays coherent while still allowing local revision.

In this sense, the machine is *codynamic*: its components evolve together rather than in isolation. The outline is not merely a planning document; it is a live structural model that guides work allocation and records the intended shape of the book. Agents do not act as independent authors; they operate as specialized workers whose tasks are derived from that structure and whose outputs are checked against it. Dependencies are not hidden implementation details; they are explicit relationships that determine what can be drafted, revised, verified, or compiled next.

The same principle applies to artifacts. A section draft is not final text until it has been reconciled with the surrounding work object, the registered references, any required diagrams, and the build pipeline that turns source into publication output. When one artifact changes, the change can propagate through the rest of the system: a revised outline can alter section work, a new section draft can expose a missing dependency, a reference update can change citation keys, and a compilation failure can reveal a mismatch between source and generated output. The machine is designed to keep those feedback loops visible rather than suppress them.

This is the practical meaning of codynamic coordination. The system does not assume that intent, structure, prose, and publication are separate phases with clean handoffs. Instead, it treats them as coupled states of one evolving work. The outline expresses intent, the agents enact it, the dependencies preserve order, the artifacts record progress, and the compiled outputs verify that the whole remains buildable. By keeping those elements synchronized, the machine can support long-form writing without losing the relationship between plan, process, and result.

That coupling is also what makes the system resilient. If the book changes direction, the outline can be revised and the downstream work can be re-evaluated. If a section needs more evidence, the reference workflow can be brought into alignment before the prose is finalized. If a diagram or other media asset is required, the request can be routed into the same coordination fabric rather than handled as an afterthought. In each case, the machine preserves a single evolving project rather than a collection of disconnected files.

A simple way to state the thesis is this: the Codynamic Book Machine works because it keeps the whole book in motion as a coordinated system. It does not merely generate text; it maintains the relationships that make the text trustworthy, buildable, and consistent with the author's intent. That is why the machine is best understood not as a pipeline with isolated stages, but as a codynamic system in which structure, labor, artifacts, and output continuously shape one another.

Put another way, codynamic behavior means that no layer is treated as permanently upstream or downstream. The outline can be revised in response to drafting realities, the drafting process can expose structural gaps, and the compiled output can validate or invalidate assumptions made earlier in the workflow. The system therefore behaves less like a one-way factory and more like a managed ecology of mutually adjusting parts. That is the thesis of the machine: coordination is not a wrapper around writing; it is the condition that makes the writing possible.

3 The Canonical Work Object

3.1 Schema, Intent, and Metadata

The Codynamic Book Machine begins by capturing the *schema* of a work before it generates any prose. That choice is not merely administrative. In a multi-agent writing system, the earliest representation of a book must already answer the questions that later agents will need in order to act consistently: What is this work for? Who is it for? What voice should it use? What publication context will it live in? Which artifacts belong to it? Without those answers, generation becomes guesswork, and guesswork is expensive once drafts, citations, diagrams, and builds begin to accumulate.

The canonical work object therefore stores intent alongside structure. Intent fields describe the purpose of the project, the intended audience, the reader takeaway, the authorial voice, and the genre or writing style. These fields are not decorative metadata; they are operational constraints. A section drafted for researchers should sound different from one drafted for general readers. A system described as a technical exposition should emphasize mechanisms, data flow, and design tradeoffs rather than narrative flourish. By recording intent up front, the machine gives downstream agents a stable target for tone, scope, and level of detail.

In practice, this means the work record can carry values such as `purpose`, `audience`, `reader_takeaway`, `author_persona`, `writing_style`, and `genre`. Those fields let the system distinguish between what the book is trying to accomplish and how it should speak while doing so. A draft can then be checked against the declared brief instead of against an informal memory of the brief. That distinction matters because the same outline can support very different books depending on whether the intended reader is a researcher, a tool builder, or a general technical audience.

Metadata serves a similar function at the publication layer. Version, creation and update timestamps, language, license, maintenance responsibility, and status all help the system distinguish a working draft from a publishable artifact. Publication settings such as document class, paper size, and front-matter options also belong in the canonical record because they affect compilation and output behavior. In other words, the work object is not just a content container; it is the source of truth for how the content should be interpreted, edited, and rendered.

A simple intake flow makes this concrete. Before a book is decomposed into chapters and sections, the system asks for the title, purpose, audience, reader takeaway, voice, genre, and design expectations, then normalizes those answers into the canonical work object. Once that happens, later agents no longer have to infer intent from prose alone. They can read it directly from the schema and act on it consistently. The same principle applies to publication metadata: a draft that is still under active revision should not be treated as a finished release, and a work intended for a book-class PDF should not be compiled as if it were a slide deck or article.

Capturing this information before generation also improves coordination. Section agents can read the same intent fields and produce prose that remains aligned across chapters. Gardener and design agents can evaluate whether a draft matches the declared audience and style. Reference and diagram workflows can be triggered only when the section context indicates that they are needed. The result is a system in which authorial intent is explicit, machine-readable, and available to every participant in the workflow.

This schema-first approach also reduces drift. If purpose, audience, and publication metadata are embedded in the work object, then later revisions do not have to reconstruct them from scattered notes or informal memory. The machine can validate the work against its declared shape, preserve continuity across agent passes, and compile outputs that remain faithful to the original brief. In that sense, schema is not a preliminary formality; it is the contract that makes codynamic authoring possible.

Publication metadata matters for the same reason. By storing status, language, license, and output settings in the work record, the system keeps editorial intent and rendering intent in the same place. That makes the work object a practical control surface rather than a passive archive. It also gives the runtime a reliable way to distinguish a draft from a release candidate, and a book from some other output form, before any expensive generation or compilation work begins.

In short, schema comes first because every later step depends on it. Intent tells the machine what to say, metadata tells it how to package and maintain the result, and the canonical work object gives every agent a shared source of truth. The rest of the system—outline decomposition, section drafting, review, diagrams, references, and compilation—can only remain coherent if they all begin from that same explicit foundation.

3.2 Recursive Outline Trees

3.2.1 Depth Without Special Cases

The recursive outline model does not need a different data structure for each level of the book. A chapter, a section, and a subsection are all instances of the same structural element, distinguished by their position in the tree rather than by special-case logic. This is the central advantage of the canonical work object: depth is represented by recursion, not by a separate schema for every tier.

In practice, the same node shape can carry the fields needed at any level of the hierarchy. A chapter node may contain section nodes in its `content` array; a section node may contain subsection nodes; and a subsection node may either continue the tree or terminate it as a leaf. The authoring system does not need one representation for "top-level content" and another for "nested content."

Instead, it applies one recursive pattern repeatedly until the outline is complete.

This uniformity matters for both humans and machines. For humans, it makes the outline easier to read because every level follows the same structural grammar. For machines, it simplifies traversal, validation, and transformation. A renderer can walk the tree with the same algorithm at every depth, collecting titles, numbers, dependencies, and content files without branching into separate code paths for chapters, sections, and subsections.

The same design also makes the outline easier to extend. If a chapter later needs another layer of subdivision, the system does not require a new schema migration or a new family of node types. It only requires another recursive step in the same model. That means the structure can grow from a short chapter list into a deeply nested book while preserving the same rules at every level.

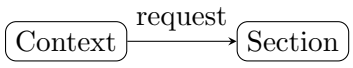
This is why the outline can treat depth as a property of position rather than a property of type. The node at the root, the node one level down, and the node several levels down all obey the same structural contract; only their parent-child relationships differ. In other words, the model does not ask whether a node is "really" a chapter or a subsection. It asks where the node sits in the tree and what children it may contain.

That consistency is what allows the Codynamic Book Machine to treat outline depth as a single recursive pattern from chapter to section to subsection and beyond. The result is a depth model without special cases, where hierarchy is a general property of the work rather than a set of exceptions handled one by one.

In implementation terms, this also means that the same validation and rendering rules can be reused at every level. A node can be checked for required identifiers, titles, dependency metadata, and child content using one schema-driven pass, regardless of whether it appears under `work.structure` as a chapter, section, or subsection. The system therefore gains consistency not only in the outline itself but also in the code that reads, verifies, and emits it.

A practical benefit of this approach is that the section model stays stable even as the manuscript grows. The same fields that describe a chapter can also describe a subsection, so the system does not need to reinterpret the meaning of a node when it moves deeper into the tree. That stability makes it easier to generate, inspect, and revise the outline because every node participates in the same structural contract.

The result is a book architecture in which depth is not a special condition to be handled separately. It is simply another consequence of recursion, and recursion is the same mechanism everywhere in the outline.



Conceptual diagram for 'Depth Without Special Cases': Show how the same structural element supports chapter, section, and subsection nodes.

Figure 2: Conceptual diagram for 'Depth Without Special Cases': Show how the same structural element supports chapter, section, and subsection nodes.

3.2.2 Structural and Narrative Dependencies

Structural dependencies encode the machine-readable edges that determine how one node in the outline relies on another. In the canonical work object, these edges are explicit data rather than implied reading order: a section can declare that it *builds on* a prior chapter, that it *depends on* a specific subsection, or that it is otherwise constrained by another part of the tree. This makes the outline actionable for validation, scheduling, and generation, because the system can inspect the dependency graph before it assigns work or compiles output.

The practical value of structural dependencies is that they separate *where content lives* from *what must already exist*. A node may be nested under a chapter for organizational purposes while still depending on material introduced elsewhere. That distinction allows the outline to remain recursive and uniform without forcing the author to encode every relationship as a parent–child relationship. In other words, the tree gives the book its shape, while dependency edges give the system its execution order.

A simple example makes the distinction concrete. A subsection may appear inside Chapter 2 because it belongs in that part of the book, yet its structural dependency may point back to Chapter 1 if it relies on terminology or a conceptual frame established there. The nesting tells the reader where the subsection belongs in the manuscript; the dependency edge tells the machine when it is safe to generate, validate, or revise that subsection.

Narrative dependencies serve a different purpose. They are human-readable notes that explain why a section comes next, what conceptual bridge it provides, or what assumptions the reader is expected to carry forward. Where structural dependencies are optimized for machines, narrative dependencies are optimized for authors and reviewers. They help answer questions such as: What prior idea does this subsection rely on? What terminology has already been established? What transition does this section provide between two larger topics?

Used together, the two forms of dependency keep the outline both precise and legible. Structural edges support automated checks, task planning, and safe generation order. Narrative notes preserve authorial intent and make the outline easier to read as a design document. The result is a dependency model that is simultaneously operational and explanatory: one layer tells the system what must happen, and the other tells the reader why the sequence makes sense. In that sense, dependencies are not just bookkeeping; they are part of the book's control surface, linking editorial intent to executable structure.

A useful implementation pattern is to keep the two dependency types distinct even when they refer to the same neighboring material. The structural edge can remain a compact machine field such as `builds_on` or `depends_on`, while the narrative note can explain the editorial reason in a sentence or two. That separation lets the runtime validate prerequisites without having to parse prose, while still giving human readers enough context to understand why the outline is arranged as it is.

This distinction also helps during revision. If a section is moved, split, or reordered, the structural dependency graph can be updated mechanically, while the narrative note can be rewritten to preserve the authorial explanation. The machine then knows what changed in the execution model, and the reader still sees a coherent account of the section's role in the larger argument. In that way, dependency metadata supports both coordination and communication.

The broader lesson is that an outline is not only a hierarchy of topics. It is also a record of constraints and transitions. Structural dependencies make those constraints explicit for the system; narrative dependencies make those transitions explicit for the author. Together they keep the manuscript both buildable and readable.

4 From Intake to Outline

4.1 Conversational Intake

Conversational intake is the system's first structured conversation with the author. Its purpose is to turn a vague project idea into a book-shaped brief that can be normalized, validated, and handed to the rest of the machine. Rather than asking the author to produce a complete outline up front, the intake process asks a small set of targeted questions that capture the minimum information needed to begin: the title, the purpose of the work, the intended audience, the reader takeaway, the desired voice, the genre, the design specification, and the expected artifact structure.

These questions are not arbitrary metadata. Each one constrains a different part of the downstream workflow. The title gives the project a stable identity. The purpose states what the book is trying to accomplish. The audience and reader takeaway define who the book is for and what success looks like from the reader's point of view. Voice and genre establish the rhetorical and stylistic frame so that later drafting can remain consistent. The design specification describes how the book should be shaped as an artifact, including any structural or presentation requirements that affect generation and compilation. Artifact structure tells the system what kinds of outputs are expected and how they should be organized.

Taken together, the question bank functions as a compact schema for author intent. It is not trying to capture every detail of the finished book; it is trying to capture the minimum set of commitments that make the rest of the pipeline reliable. A title without purpose is only a label. A purpose without audience is only an aspiration. An audience without reader takeaway does not yet define success. The intake conversation therefore asks for these fields in combination so that the system can build a coherent project model rather than a loose collection of preferences.

The question bank also separates discovery from drafting without separating them completely. The author is not forced to solve the whole book in one pass, but the system still receives enough information to normalize the project into machine-readable fields, check for missing pieces, and seed the outline and section workflow that follows. In practice, conversational intake behaves like a guided interview and a schema-filling step at the same time. It is conversational because it helps the author think through the project. It is operational because each answer becomes part of the canonical work object.

That dual role matters for the rest of the machine. Once the intake answers are recorded, later agents can rely on them as stable context instead of repeatedly inferring intent from free-form prose. The outline can be shaped around the stated purpose. Drafting can follow the declared voice and genre. Design and assembly can respect the artifact structure from the start. In this way, the intake stage is not merely administrative overhead; it is the point where author intent becomes operational structure.

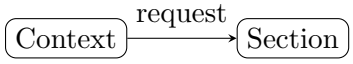
The reader takeaway from this stage is simple: before the machine can write, it must know what it is writing, for whom, and in what form. Conversational intake supplies that information in a controlled way, so the rest of the pipeline can operate on a coherent project rather than an ambiguous prompt.

In the broader workflow, this section sits at the boundary between human intention and machine-readable structure. It is the first place where the system asks the author to commit to the constraints that will govern later normalization, section generation, and artifact assembly. That is why the question bank is small but specific: it captures the minimum viable brief needed to make the rest of the book process dependable.

A practical intake sequence usually begins with the title and purpose, then moves to audience and reader takeaway, and finally resolves voice, genre, design specification, and artifact structure.

That ordering is deliberate. Early answers establish what the project is about; later answers refine how the system should present it. If the author is uncertain about one field, the conversation can still proceed by recording a provisional answer and revisiting it after the rest of the brief is clearer. The goal is not perfection on the first pass. The goal is to collect enough structured intent to start the machine without losing the author's original direction.

For that reason, the question bank is best understood as a controlled handshake between author and system. The author supplies intent in natural language, and the machine converts that intent into a stable brief that downstream agents can trust. Once that handshake is complete, the rest of the workflow can treat the project as a defined artifact rather than an open-ended conversation.



Conceptual diagram for 'Conversational Intake': Explain the question bank: title, purpose, audience, reader takeaway, voice, genre, design spec, and artifact structure.

Figure 3: Conceptual diagram for 'Conversational Intake': Explain the question bank: title, purpose, audience, reader takeaway, voice, genre, design spec, and artifact structure.

4.2 Outline Import and Normalization

This section explains how the machine accepts legacy outlines and turns them into the canonical work object used everywhere else in the system. The importer is intentionally conservative: it does not try to preserve every surface detail of a source format. Instead, it extracts structure, intent, and content, then normalizes those elements into the same schema that conversational intake produces.

The supported inputs reflect the kinds of material authors already have on hand. A project may begin as legacy YAML, a Markdown document with heading structure, a numbered outline, or even a block of raw text. Each of these forms carries useful information, but each encodes that information differently. The importer therefore performs two related tasks: it parses the source format, and it maps the parsed result into canonical work YAML.

For legacy YAML, the importer reads the existing keys and compares them against the current work schema. Fields that already match the canonical model can be carried forward directly. Fields that are older, renamed, or nested differently are translated into the current structure. This makes migration possible without forcing authors to rewrite a project by hand before the system can use it. When the source file is close to the current schema, the importer behaves like a translator; when it is older or partially incompatible, it behaves like a migration layer.

Markdown imports are treated as structural documents. Heading levels become outline nodes, paragraph text becomes section content, and list structure is preserved when it contributes to meaning. The importer uses the heading hierarchy to infer parent–child relationships, so a document organized with `#`, `##`, and deeper headings can be converted into a recursive outline tree. The goal is not to reproduce Markdown as Markdown, but to recover the authorial structure that the headings imply. In practice, this means the importer treats headings as signals of hierarchy rather than as formatting to be preserved verbatim.

Numbered outlines are normalized in a similar way. A sequence such as `1.`, `1.1`, and `1.1.1`

already expresses hierarchy, but the numbering itself is not the canonical representation. The importer uses the numbering to determine nesting, then stores the result as structured nodes with stable identifiers and content fields. Once normalized, the outline no longer depends on the visual numbering scheme that produced it. This is important because the canonical work object must remain stable even if the author later renumbers or reorders the source outline.

Raw text is the least structured input, so the importer applies the most cautious treatment. It can preserve the text as content, but it cannot reliably infer deep hierarchy unless the source includes explicit cues such as blank-line separation, repeated labels, or numbering. In practice, raw text imports are often used as a starting point for manual refinement: the importer captures what it can, then leaves the resulting work object ready for later editing. That restraint is deliberate, because false structure is more damaging than missing structure.

Across all input types, normalization serves the same purpose: it produces a single canonical work YAML representation rooted at `work`. That representation can then be validated, diffed, routed through agents, and compiled without special cases for the original source format. The importer therefore acts as a boundary between heterogeneous author input and the machine's shared internal model.

This normalization step is also where migration reports become useful. When the importer encounters missing fields, ambiguous hierarchy, or source elements that do not map cleanly into the canonical schema, it can record those issues for review. The result is a work object that is usable immediately, plus a record of what was inferred, translated, or left unresolved. In that sense, import is not merely conversion; it is the first disciplined act of editorial reconciliation in the system.

A simple example makes the distinction concrete. A legacy outline might arrive as a Markdown file with headings and paragraphs, or as YAML with nested sections already partially described. The importer does not ask the author to choose a new format first. It reads the source as it is, identifies the structural cues available in that source, and emits the same canonical shape in both cases. That shared shape is what allows later stages of the system to work uniformly.

By the end of this process, the project is no longer a legacy file, a loose outline, or an unstructured note. It is a canonical work object that the rest of the Codynamic Book Machine can understand consistently. From that point forward, the book can move through validation, decomposition, agent work, and compilation without needing to remember how it began.

A useful way to think about the importer is as a normalization funnel. At the top of the funnel, the system accepts several familiar authoring forms; at the bottom, it emits one machine-readable work structure. The narrower output is not a loss of meaning so much as a reduction of ambiguity. The importer keeps the distinctions that matter for later processing, such as section boundaries, nesting, and content text, while discarding source-specific presentation details that would otherwise complicate downstream tooling.

That design also keeps the rest of the pipeline simpler. Once import has produced canonical work YAML, later components do not need separate code paths for Markdown, YAML, numbered notes, or freeform text. They can operate on one representation and rely on the importer to have already resolved the differences among source formats. In a system built around durable coordination, that early standardization is what makes the rest of the workflow predictable.

4.3 Book Repository Layout

The repository layout is the operational map of the book machine: it separates what the author intends to say from what the system generates, records, and compiles. That separation keeps the project inspectable at every stage, because each class of artifact has a predictable place and a distinct purpose.

At the top level, the repository holds the canonical outline and the working content tree. The outline files describe the book structure, section order, and dependency relationships. The content files hold the prose for each section, usually as source material or normalized section notes before they are rendered into final LaTeX. This division matters because the outline is the control plane, while the section payloads are the writing surface.

Section drafts are stored as LaTeX payloads in per-section files. Each payload corresponds to one section identifier, which makes it possible to regenerate or revise a single section without rewriting the entire book. In practice, this means the authoring system can route work to a section agent, receive a LaTeX body, and write that body to the section payload path associated with the section id. The payloads are therefore the durable draft layer of the repository.

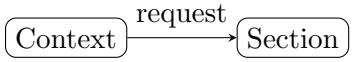
The repository also contains logs and runtime traces. These records capture intake events, normalization steps, agent prompts, task assignments, and revision history. Logs are not part of the prose itself, but they are essential for understanding how a draft came to exist. They provide the audit trail that lets the system explain why a section changed, which agent touched it, and what dependency or blocker shaped the work.

Build artifacts occupy a separate area of the repository. These include compiled outputs, intermediate files, and any generated products created during document assembly. Keeping build artifacts distinct from source content prevents accidental editing of derived files and makes it easier to clean, rebuild, or compare outputs across runs. The source tree remains authoritative, while the build tree remains disposable and reproducible.

Media requests are tracked as first-class artifacts rather than informal notes. When a section needs a figure, diagram, or other visual asset, the request is recorded with enough structure for another agent to fulfill it. The section agent does not invent a file path or silently embed a missing asset; instead, it requests the media through the coordination layer and waits for a returned path and inclusion hint. This keeps visuals synchronized with the text and prevents broken references.

An artifact registry ties these pieces together. It records the generated outputs, their locations, and the relationships among source files, section payloads, media assets, and compiled deliverables. The registry gives the system a single place to answer questions such as which section produced a given LaTeX file, which diagram belongs to which request, or which build artifact corresponds to a particular revision. In that sense, the registry is the repository's index of durable work.

Taken together, the outline files, content sections, LaTeX payloads, logs, build artifacts, media requests, and artifact registry form a layered repository architecture. Each layer has a narrow role, but the layers remain linked through identifiers and recorded events. That structure is what allows the Codynamic Book Machine to treat a book not as a single monolithic file, but as a coordinated set of evolving artifacts.



Conceptual diagram for 'Book Repository Layout': Describe outline files, content sections, TeX payloads, logs, build artifacts, media requests, and artifact registry.

Figure 4: Conceptual diagram for 'Book Repository Layout': Describe outline files, content sections, TeX payloads, logs, build artifacts, media requests, and artifact registry.

5 Agents, Work, and Coordination

5.1 Agent Roles

Agent roles in the Codynamic Book Machine are divided so that each responsibility has a clear owner, a narrow interface, and a predictable place in the workflow. The result is not a single monolithic authoring process but a coordinated set of specialized tasks that move a book from intent to outline, from outline to draft, and from draft to compiled artifact.

At the center of the system is the hypervisor. The hypervisor does not write prose itself; instead, it supervises the overall run, assigns work, and keeps the project moving through the correct phases. It is responsible for initiating tasks, routing work to the appropriate agent, and maintaining the high-level view of progress across the book. In practice, the hypervisor acts as the orchestration layer that ensures the rest of the system is working on the right section at the right time.

The outlining agent turns author intent and imported material into a structured book plan. Its job is to shape the canonical outline, organize chapters and sections, and preserve the dependencies that make later drafting possible. Where the hypervisor manages the run, the outlining agent manages the structure of the work itself. It establishes the sequence of topics and the relationships among them so that downstream agents can draft against a stable scaffold.

Section drafting agents are responsible for producing the
LaTeX body of individual sections. Each section agent works within a bounded payload and converts outline intent, imported notes, and coordination feedback into prose. This division keeps drafting local and incremental: a section agent does not need to solve the whole book, only the section it owns. That constraint makes revision easier and allows the system to parallelize work across multiple sections when needed.

The gardener agent performs cleanup and consistency work. It responds to structural or stylistic issues that emerge after drafting, such as uneven transitions, duplicated ideas, missing connective tissue, or mismatches between neighboring sections. Rather than introducing new subject matter, the gardener refines what is already present so the manuscript reads as a coherent whole. In this sense, gardening is the maintenance layer of the writing process.

Diagram agents handle visual assets when a figure, chart, or schematic would materially improve understanding. Their role is to create or retrieve the media needed to support the text and to return a usable path and inclusion hint for the
LaTeX source. This keeps visual production separate from prose drafting while still allowing the section to request a diagram when the argument benefits from one.

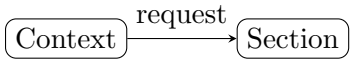
The design agent manages presentation-level concerns that affect the appearance and layout of the book. It is concerned with how the manuscript is rendered, how structural choices interact with the document format, and how the visual system supports readability. Where the diagram agent supplies content-specific visuals, the design agent helps ensure that the broader document design remains consistent.

The references agent owns citation registration and bibliography coordination. When a section needs a source that is not yet available in the shared reference set, the section workflow must request research and then submit candidate records to the references agent. This separation prevents ad hoc bibliography edits and keeps citation keys canonical across the project. The references agent therefore serves as the gatekeeper for source integrity and citation consistency.

Finally, the assembly agent compiles the finished manuscript into publication artifacts. It gathers the drafted sections, resolves the document structure, and produces the outputs used for review and release. Assembly is the point at which the distributed work of the other agents becomes a single book-shaped artifact.

Taken together, these roles form a pipeline with explicit boundaries: hypervision coordinates the run, outlining defines structure, section agents draft content, gardening improves coherence, diagram and design agents support presentation, the references agent controls source registration, and assembly produces the final output. The system is deliberately modular, but the modules are not independent. Each role exists to reduce ambiguity for the next role, so that the book can advance through a disciplined sequence of transformations rather than a loose collection of edits.

This division of labor also clarifies how the machine should be extended. New capabilities are added by assigning them to a role with a well-defined responsibility, rather than by expanding every agent's scope at once. That keeps the workflow legible: each agent knows what it owns, what it may request from others, and when its work is complete. In a long-form authoring system, that clarity is not just an implementation detail; it is what makes coordinated writing possible.



Conceptual diagram for 'Agent Roles': Map the major agents to their responsibilities: hypervision, outlining, section drafting, gardening, diagrams, design, references, and assembly.

Figure 5: Conceptual diagram for 'Agent Roles': Map the major agents to their responsibilities: hypervision, outlining, section drafting, gardening, diagrams, design, references, and assembly.

5.2 Work Management

Work management is the layer that turns a stream of authoring intent into bounded, observable labor. In the Codynamic Book Machine, the unit of scheduling is the *WorkItem*: a discrete task with a clear purpose, an owner or target agent, a status, an estimate of effort, and enough context to resume work without reconstructing the entire conversation. WorkItems are used for drafting, review, diagram requests, reference lookup, cleanup, and assembly tasks, but they all share the same operational shape: they can be queued, blocked, checkpointed, and completed.

A *WorkManager* coordinates these items across the system. Its role is not to perform the work itself, but to maintain the queue discipline that keeps the machine coherent. It accepts new items, orders them by priority and dependency, tracks which items are active, and records when an item is waiting on another item, a missing artifact, or a human decision. In this sense, the WorkManager is the operational counterpart to the canonical work object: the work object describes what the book is, while the WorkManager describes what must happen next.

A useful way to understand this layer is to follow a WorkItem through its lifecycle. An item may be created from an outline requirement, a revision request, or a coordination need. It enters the queue with a priority and a dependency set, becomes active when capacity is available, and either advances to completion or pauses in a blocked state if it cannot proceed. At each meaningful boundary, the item can be checkpointed so that its current state is preserved without forcing the agent to repeat earlier reasoning.

Checkpoints are the mechanism that makes long-running work resumable. A checkpoint captures the current state of a WorkItem at a meaningful boundary: what has been completed, what remains, what assumptions were made, and what external inputs are still pending. For section

drafting, a checkpoint may include the current LaTeX body, unresolved citation needs, open diagram requests, and any editorial constraints inherited from the outline. For coordination tasks, a checkpoint may record the last known message state or the latest decision from the hypervisor. Checkpoints reduce the cost of interruption and make it possible to hand work between agents without losing continuity.

Capacity reports provide the system with a simple but important form of self-observation. A capacity report summarizes how much work an agent or queue can reasonably accept at a given moment, how many items are active, what kinds of tasks are consuming attention, and whether the current load is sustainable. This is especially important in a multi-agent environment, where the appearance of parallelism can hide a practical bottleneck. If one agent is overloaded with high-priority items, the report should make that visible before the queue becomes unstable.

The WorkManager also tracks blockers explicitly. A blocker is any condition that prevents a WorkItem from advancing: a missing reference key, an unresolved dependency, a required diagram that has not yet been generated, a conflicting instruction, or a downstream artifact that has not been registered. Blockers are not treated as vague delays; they are first-class state. That distinction matters because it allows the system to distinguish between work that is merely waiting and work that is structurally impossible to complete until another action occurs.

Overcommitment is the failure mode that queue discipline is designed to prevent. When too many items are accepted at once, the system may appear productive while actually accumulating hidden debt: partially drafted sections, unresolved citations, duplicated requests, and inconsistent revisions. The WorkManager therefore needs to enforce limits on concurrent work and to surface when the queue exceeds available capacity. Overcommitment is not only a scheduling problem; it is a quality problem, because rushed coordination tends to produce artifacts that are harder to verify and revise.

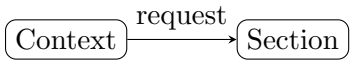
Queue discipline is the rule set that keeps this from happening. Items should enter the queue with explicit priority and dependency information, should move forward only when their prerequisites are satisfied, and should be marked blocked rather than silently stalled. The queue should prefer small, finishable units of work over broad, ambiguous ones, because smaller items are easier to checkpoint, easier to review, and easier to reassign. In practice, this means the system favors a steady flow of completed WorkItems over a large backlog of partially addressed ones.

Taken together, WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline define the operational grammar of the machine. They ensure that authoring is not just a sequence of text generations, but a managed process in which progress is visible, interruptions are survivable, and coordination remains bounded by explicit state. In the next section, that operational grammar extends into the message layer, where routed prompts, durable logs, and runtime memory make the coordination records themselves persistent.

5.3 Messaging and Runtime Memory

Messaging is the connective tissue of the Codynamic Book Machine. If work items define what should happen, messages define how the system announces, routes, records, and replays what is happening. The runtime treats coordination as a stream of explicit events and task prompts rather than as hidden state in a single controller. That design makes agent activity inspectable, recoverable, and easier to resume after interruption.

At the center of this layer is routed messaging. A routed message is not merely a note sent from one agent to another; it is a structured record that names the sender, the intended recipient or topic, the payload, and the reason for delivery. Routing allows the system to separate message production from message consumption. An agent can emit a request, a status update, a blocker



Conceptual diagram for 'Work Management': Explain WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline.

Figure 6: Conceptual diagram for 'Work Management': Explain WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline.

report, or a completion notice without needing to know which component will process it next. The router resolves that intent into the appropriate queue, subscription, or coordination channel.

Subscriptions provide the receiving side of that arrangement. Agents and services subscribe to the message types or topics relevant to their role, so the hypervisor can observe lifecycle events, the work manager can track task transitions, and specialized agents can react to prompts that match their responsibilities. This keeps the runtime loosely coupled: a section agent does not need direct knowledge of the references agent's internal implementation, only the message contract that connects them. In practice, subscriptions also make it possible to add new observers, such as logging or auditing components, without changing the agents that produce the messages.

The system persists important messages in durable logs. A durable log is the memory of the runtime: it records what was requested, what was acknowledged, what was completed, and what failed. This persistence matters because the book machine is not a one-shot generator. It revisits sections, revises drafts, and coordinates multiple agents over time. Durable logs allow the system to reconstruct the sequence of actions that led to a given artifact, which is essential for debugging, review, and recovery after a restart. They also provide a stable history that can be inspected when a section appears inconsistent with its dependencies or when a task seems to have vanished from the active queue.

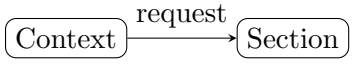
Task prompts are the operational form of these messages. A prompt packages the instruction an agent should execute, along with the context needed to do it well: the section identifier, the current phase, the requested action, and any constraints or references that apply. In the Codynamic Book Machine, prompts are not informal chat messages; they are work directives. A prompt can ask a section agent to draft prose, a gardener to revise for consistency, a diagram agent to create a figure, or a references agent to register citations. Because prompts are explicit and structured, they can be stored, replayed, and compared against the resulting output.

The runtime registry is the system's live map of agents and their state. It records which agents exist, what roles they are currently playing, what tasks they are assigned, and what lifecycle stage they occupy. This registry gives the hypervisor and work manager a shared view of the active system. It is the difference between knowing that an agent exists in principle and knowing that it is currently available, busy, blocked, or finished. The registry also supports coordination across phases: when a task is handed off from drafting to review, the runtime can update the agent record and preserve the continuity of that work.

Coordination records tie the messaging layer back to the book itself. These records capture the relationships among prompts, responses, checkpoints, and artifacts so that the system can explain why a section changed and which agent actions produced the change. A coordination record may note that a section draft was generated, then revised after gardener feedback, then held pending

citation registration, or that a diagram request was issued and later satisfied by an external asset path. In this sense, coordination records are the narrative trace of the machine's operation.

Taken together, routed messages, subscriptions, durable logs, task prompts, the agent runtime registry, and coordination records form the machine's working memory. They let the system distribute labor without losing accountability. They also make the authoring process legible: every important transition can be traced from intent to prompt, from prompt to response, and from response to durable record. That traceability is what allows the Codynamic Book Machine to behave less like a black box and more like a disciplined collaborative system.



Conceptual diagram for 'Messaging and Runtime Memory': Explain routed messages, subscriptions, durable logs, task prompts, agent runtime registry, and coordination records.

Figure 7: Conceptual diagram for 'Messaging and Runtime Memory': Explain routed messages, subscriptions, durable logs, task prompts, agent runtime registry, and coordination records.

6 Drafting, Review, and Artifacts

This chapter explains the point at which generated text stops being a transient draft and becomes a reviewable, durable artifact. In the Codynamic Book Machine, authoring is not a single act of composition followed by a final export. It is a staged process in which content is drafted, checked, revised, and then committed into files and build outputs that can be inspected independently of the agent that produced them.

The distinction matters because long-form work accumulates dependencies. A section draft may depend on outline structure, citation keys, diagram assets, and prior chapter claims. If those dependencies remain implicit, the result is fragile: a text may read well in isolation but fail during compilation, drift from the canonical outline, or become difficult to audit later. The machine therefore treats review and artifact creation as first-class steps in the authoring pipeline rather than as incidental cleanup.

Drafting produces the first reviewable representation of a section. At this stage, an agent translates outline intent into prose, preserves the section's scope, and writes the result into the section payload file. The draft is intentionally local: it should be understandable on its own, but it is still only one node in a larger work graph. A good draft makes its assumptions visible, uses the established terminology of the book, and leaves room for later refinement by gardeners, reviewers, and reference agents.

Review turns a draft into a controlled object. Review checks whether the section matches its outline goal, whether it introduces unsupported claims, whether it uses registered citation keys, and whether it refers to diagrams or other media that actually exist. It also checks continuity across neighboring sections so that repeated definitions, transitions, and terminology remain consistent. In this system, review is not merely editorial preference; it is a coordination mechanism that keeps the book's structure aligned with its content.

Artifacts are the durable outputs of that coordination. The section payload file is one artifact, but it is not the only one. Build logs, generated TEX, compiled PDF output, media assets, and reference records all become part of the book's working memory. These artifacts allow the system to answer practical questions later: what was generated, when it was generated, which inputs were used, and whether the output compiled successfully. Durability here means that the work can be resumed, audited, or rebuilt without reconstructing the entire authoring history from scratch.

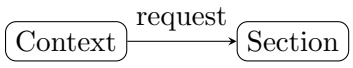
A useful way to think about the lifecycle is as a boundary crossing. Before review, a section is a proposal: it may be incomplete, provisional, or dependent on unresolved tasks. After review and artifact generation, it becomes a record that other agents and human readers can rely on. That record is not immutable in the absolute sense; it can still be revised. But revisions happen against a stable baseline, with the previous state preserved in files, logs, or version control rather than lost inside an agent's transient context.

This chapter therefore focuses on the transition from mutable work to stable record. Drafts are meant to change; artifacts are meant to persist. The machine succeeds when it can move content across that boundary without losing traceability, without breaking compilation, and without severing the relationship between the prose a reader sees and the structured work that produced it.

A practical implication of this design is that review is not a separate after-the-fact reading pass. It is part of the same workflow that creates the draft in the first place. The section agent writes a draft, the surrounding coordination layer checks whether the draft satisfies the outline and the current dependency state, and any unresolved issues are pushed back into the queue as explicit work. That makes the section payload a living artifact: it can be inspected, revised, and compared, but it is always tied to a specific state of the project.

The same principle applies to compilation. A successful build is not just a convenience; it is evidence that the manuscript has reached a coherent state. If the source compiles, the system has at least verified that the section structure, included assets, and registered references are mutually compatible at that moment. If the build fails, the failure is itself an artifact, because it records a mismatch that must be resolved before the work can be treated as durable.

Seen this way, the chapter's subject is not only writing, but stabilization. The machine drafts to discover what the section should say, reviews to test whether it says it correctly, and emits artifacts so that the result can survive beyond the current agent run. That progression is what turns generated prose into book infrastructure.



Conceptual diagram for 'Drafting, Review, and Artifacts': Explain how generated work becomes reviewable and durable.

Figure 8: Conceptual diagram for 'Drafting, Review, and Artifacts': Explain how generated work becomes reviewable and durable.